



Algoritmos de ordenação



Prof. Rafael Mesquita
rgm@cin.ufpe.br

O que veremos nesta aula?

- Introdução
- Algoritmos de ordenação e suas complexidades
 - ▣ Bubble Sort
 - ▣ Mergesort
 - ▣ Quicksort

Introdução

- Definição de ordenação
 - ▣ Processo de rearranjar uma sequência de objetos de acordo com alguma sequência lógica
 - ▣ Operação fundamental na computação
 - “Nos primórdios da computação era de conhecimento comum que até 30% de todos os ciclos de processamento era gasto com ordenação
 - Hoje, essa proporção é menor, devido à eficiência dos algoritmos de ordenação” [2]
 - ▣ Grande importância em aplicações comerciais e científicas

Introdução

- Necessidade de ordenação na computação é inquestionável
- Operação fundamental em ciência da computação
 - ▣ Eficiência de diversas operações, como a de busca, dependem da ordenação
 - Ex: busca binária
 - ▣ Ordenação também influencia na eficiência de operações de busca em situações cotidianas
 - Busca por um telefone em um catálogo
 - Busca por um livro em uma biblioteca

Bubble Sort

- Classificação por bolha
 - ▣ Fácil de entender e programar
 - ▣ No entanto, é ineficiente!
- Considere um vetor x , que deve ser ordenado de forma crescente
- Idéia do método:
 - ▣ Percorrer o vetor sequencialmente várias vezes
 - ▣ Em cada passagem, cada um dos elementos do vetor é comparado com seu sucessor
 - Troca ocorre caso seja necessário (se $v[x+1] > v[x]$)

Bubble Sort

- Exemplo:
- Vetor de entrada:
 - ▣ 25 57 48 37 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 57 48 37 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 57 48 37 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 57 48 37 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 57 37 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 57 37 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 57 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 57 12 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 12 57 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 12 57 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 12 57 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 12 57 92 86 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 12 57 86 92 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 12 57 86 92 33

Bubble Sort

- Exemplo:
- Primeira passagem:
 - ▣ 25 48 37 12 57 86 33 92
 - ▣ Término da primeira passagem!
 - ▣ Note que o maior elemento (92) está na sua posição correta...
 - ▣ Generalizando
 - ▣ O elemento $v[n-i]$ ficará na sua posição correta após a iteração (passagem) i
 - $n \rightarrow$ número de elementos do vetor

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 48 37 12 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 48 37 12 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 48 37 12 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 48 37 12 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 48 12 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 48 12 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 12 48 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 12 48 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 12 48 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 12 48 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 12 48 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 12 48 57 86 33 92

BubbleSort

- Exemplo:
- Segunda passagem:
 - ▣ 25 37 12 48 57 33 86 92
 - ▣ Término da segunda passagem!
 - ▣ Último e penúltimo elementos ordenados
 - ▣ Note que não foi necessário comparação com o último elemento, pois ele já estava ordenado...

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 37 12 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 37 12 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 37 12 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 37 12 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 12 37 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 12 37 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 12 37 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 12 37 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 12 37 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 12 37 48 57 33 86 92

BubbleSort

- Exemplo:
- Terceira passagem:
 - ▣ 25 12 37 48 33 57 86 92
 - ▣ Fim da terceira passagem!

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 25 12 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 25 12 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 12 25 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 12 25 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 12 25 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 12 25 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 12 25 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 12 25 37 48 33 57 86 92

BubbleSort

- Exemplo:
- Quarta passagem:
 - ▣ 12 25 37 33 48 57 86 92

BubbleSort

- Exemplo:
- Quinta passagem:
 - ▣ 12 25 37 33 48 57 86 92

BubbleSort

- Exemplo:
- Quinta passagem:
 - ▣ 12 25 37 33 48 57 86 92

BubbleSort

- Exemplo:
- Quinta passagem:
 - ▣ 12 25 37 33 48 57 86 92

BubbleSort

- Exemplo:
- Quinta passagem:
 - ▣ 12 25 37 33 48 57 86 92

BubbleSort

- Exemplo:
- Quinta passagem:
 - ▣ 12 25 37 33 48 57 86 92

BubbleSort

- Exemplo:
- Quinta passagem:
 - ▣ 12 25 37 33 48 57 86 92

BubbleSort

- Exemplo:
- Quinta passagem:
 - ▣ 12 25 33 37 48 57 86 92

BubbleSort

- Exemplo:
- Sexta passagem:
 - ▣ 12 25 33 37 48 57 86 92

BubbleSort

- Exemplo:
- Sexta passagem:
 - ▣ 12 25 33 37 48 57 86 92

BubbleSort

- Exemplo:
- Sexta passagem:
 - ▣ 12 25 33 37 48 57 86 92

BubbleSort

- Exemplo:
- Sexta passagem:
 - ▣ 12 25 33 37 48 57 86 92

BubbleSort

- Exemplo:
- Sexta passagem:
 - ▣ 12 25 33 37 48 57 86 92
 - ▣ Poderíamos continuar o algoritmo até que não restasse nenhum elemento a ser ordenado
 - ▣ No entanto, na sexta passagem não ocorreu nenhuma troca
 - Algoritmo pode ser finalizado em qualquer passagem que não exista nenhuma troca
 - ▣ Vetor está ordenado

Bubblesort

- Complexidade

- $O(n^2)$

- Melhor caso:

- $O(n)$

- Vetor já ordenado...

- Qual o mérito de um algoritmo de classificação aplicado a um conjunto já ordenado?!

Mergesort

- Base: operação de *merge*
 - ▣ Dados dois arrays ordenados os seus elementos são combinados
 - ▣ Forma-se um único array com todos os seus elementos ordenados
- ***Dividir para conquistar!***

Mergesort

□ Merge

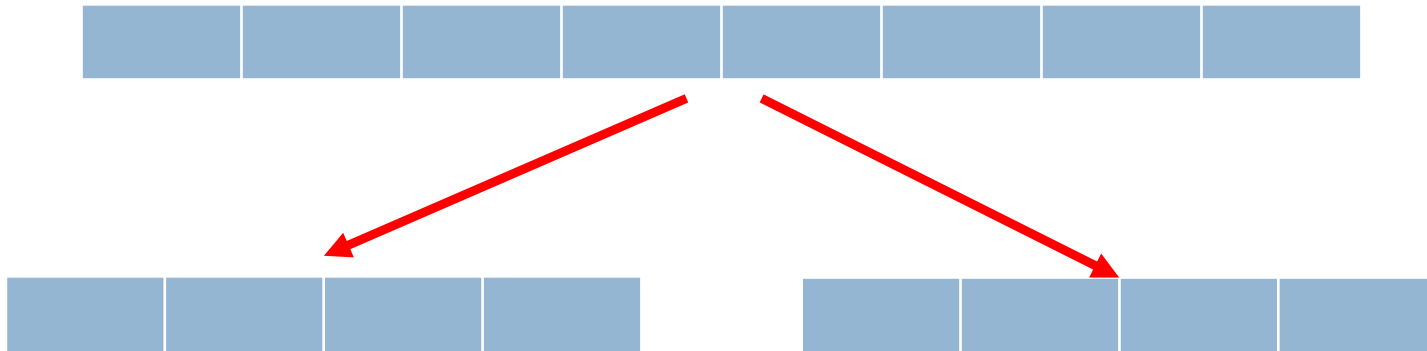
```
void merge(int vetor[], int aux[], int low, int mid, int high)
{
    int i = low;
    int j = mid + 1;

    for (int k = low; k <= high; k++)
        aux[k] = vetor[k];

    for (int k = low; k <= high; k++)
    {
        if (i > mid)
            vetor[k] = aux[j++];
        else if (j > high)
            vetor[k] = aux[i++];
        else if (aux[j] < aux[i])
            vetor[k] = aux[j++];
        else
            vetor[k] = aux[i++];
    }
}
```

Mergesort

- Implementação recursiva
 - Array é dividido em dois
 - Caso as duas metades estejam ordenadas, o conjunto completo estará ordenado após operação de merge

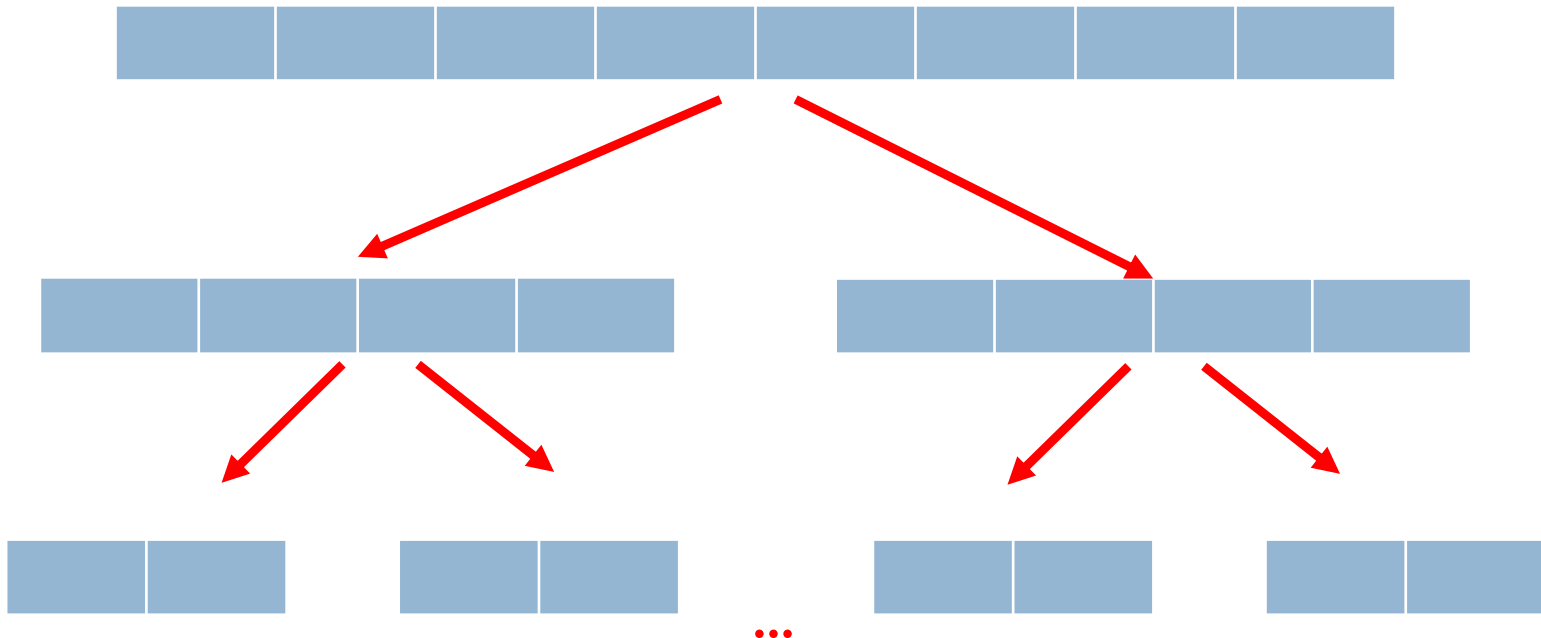


Mergesort

- Implementação recursiva

- Como garantir que duas metades estejam ordenadas?

- Aplica-se o Mergesort, recursivamente, em cada metade do vetor!



Mergesort

```
void sort(int vetor[], int aux[], int low, int high)
{
    if (low >= high)
        return;
    int mid = (low + high) / 2;

    sort(vetor, aux, low, mid);

    sort(vetor, aux, mid + 1, high);

    merge(vetor, aux, low, mid, high);
}
```

Mergesort

□ Análise eficiência assintótica

□ Tempo

- $\Theta(n \cdot \log(n))$

- Demonstração em sala!

- Mais eficiente que bubblesort, selection sort e insertion sort [2]

- Porém, espaço extra é necessário

- Vetor auxiliar

□ Espaço

- $\Theta(n)$

QuickSort

- Como o Mergesort, também é baseado no paradigma “dividir para conquistar”
- Entretanto, divisões das partições são dinâmicas
- Passos básicos:
 1. Escolhe-se um pivô
 2. O pivô é posicionado de forma que todos os elementos anteriores a ele sejam menores e todos os posteriores, maiores

Elementos menores que o pivô	pivô	Elementos maiores que o pivô
---	-------------	---

3. Recursivamente, subvetores à esquerda e à direita são ordenados

QuickSort

- Com base no item dos 2 fica claro que o pivô encontra-se na sua posição correta
 - ▣ No entanto, os subvetores à esquerda e à direita não necessariamente estarão ordenados
- Ao executar as chamadas recursivas, ordenamos os subvetores

QuickSort

- Exemplo

- 25 57 48 37 12 92 86 33

QuickSort

- Exemplo
- (**25** 57 48 37 12 92 86 33}
- **pivo**

QuickSort

- Exemplo
- (1 2) **25** (57 48 37 92 86 33)
- Pivo colocado na posição correta
 - ▣ Por enquanto, abstraímos a forma como isso é feito...
 - ▣ 25 encontra-se na posição correta
 - ▣ $(1\ 2) < 25 < (57 \dots 33)$
- Aplicamos o metodo recursivamente a cada um dos subvetores

QuickSort

- Exemplo
- (12) 25 (57 48 37 92 86 33)
 - ▣ Ordenação do subvetor à esquerda de 25 (antigo pivô)

QuickSort

- Exemplo
- (1 2) 25 (**57** 48 37 92 86 33)
- Ordenação do subvetor à direita de 25 (antigo pivô)

QuickSort

- Exemplo

- 12 25 (48 37 33) **57** (92 86)

QuickSort

- Exemplo

- 12 25 (**48** 37 33) 57 (92 86)

QuickSort

- Exemplo

- 12 25 (37 33) 48 57 (92 86)

QuickSort

- Exemplo

- 12 25 (**37** 33) 48 57 (92 86)

QuickSort

- Exemplo

- 12 25 (33) 37 48 57 (92 86)

QuickSort

- Exemplo

- 12 25 (**33**) 37 48 57 (92 86)

QuickSort

- Exemplo
- 12 25 33 37 48 57 (92 86)

QuickSort

- Exemplo
- 12 25 33 37 48 57 (92 86)

QuickSort

- Exemplo

- 12 25 33 37 48 57 (86 92)

QuickSort

- Exemplo

- 12 25 33 37 48 57 (86) 92

QuickSort

- Exemplo

- 12 25 33 37 48 57 (86) 92

QuickSort

- Exemplo
- 12 25 33 37 48 57 86 92

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ ub (limite máximo do subvetor)
 - ▣ lb (limite mínimo do subvetor)
 - ▣ $pivo = v[lb]$ (elemento cuja posição será procurada)
 - Nessa implementação, utilizamos o primeiro elemento do subvetor como pivô
 - Outras abordagens podem ser utilizadas:
 - Escolha aleatória, elemento do meio, do final, mediana de n elementos, ...
 - Necessário trocar o elemento escolhido como pivô para a posição lb

5	0	2	7	1	3	4	8	6
---	---	---	---	---	---	---	---	---

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Ponteiro down é movido de lb em direção á ub
 - todo elemento de posição abaixo de down deve ser menor ou igual ao pivô
 - ▣ Ponteiro up é movido de ub em direção á lb
 - todo elemento de posição acima de up deve ser maior ou igual ao pivô
 - ▣ Ponteiros são movidos enquanto tais condições forem verdadeiras

5	0	2	7	1	3	4	8	6
---	---	---	---	---	---	---	---	---

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Ponteiro down é movido de lb em direção á ub
 - todo elemento de posição abaixo de down deve ser menor ou igual ao pivô
 - ▣ Ponteiro up é movido de ub em direção á lb
 - todo elemento de posição acima de up deve ser maior ou igual ao pivô
 - ▣ Ponteiros são movidos enquanto tais condições forem verdadeiras

down

up

5	0	2	7	1	3	4	8	6
---	---	---	---	---	---	---	---	---

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Ponteiro down é movido de lb em direção á ub
 - todo elemento de posição abaixo de down deve ser menor ou igual ao pivô
 - ▣ Ponteiro up é movido de ub em direção á lb
 - todo elemento de posição acima de up deve ser maior ou igual ao pivô
 - ▣ Ponteiros são movidos enquanto tais condições forem verdadeiras

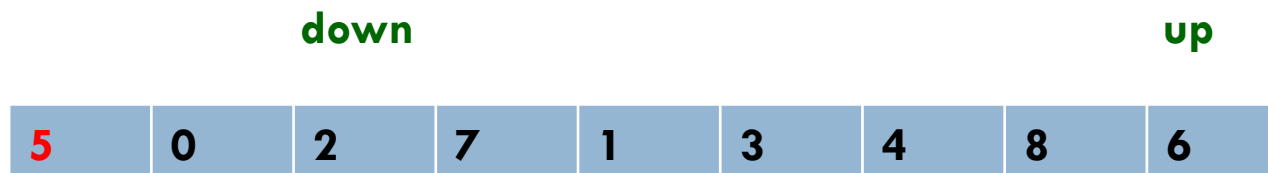
down

up

5	0	2	7	1	3	4	8	6
---	---	---	---	---	---	---	---	---

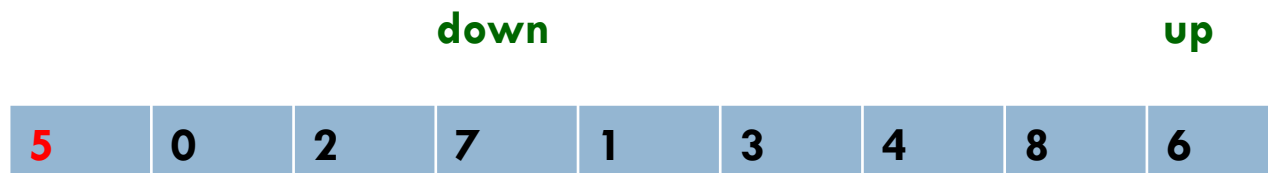
QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Ponteiro down é movido de lb em direção á ub
 - todo elemento de posição abaixo de down deve ser menor ou igual ao pivô
 - ▣ Ponteiro up é movido de ub em direção á lb
 - todo elemento de posição acima de up deve ser maior ou igual ao pivô
 - ▣ Ponteiros são movidos enquanto tais condições forem verdadeiras



QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Ponteiro down é movido de lb em direção á ub
 - todo elemento de posição abaixo de down deve ser menor ou igual ao pivô
 - ▣ Ponteiro up é movido de ub em direção á lb
 - todo elemento de posição acima de up deve ser maior ou igual ao pivô
 - ▣ Ponteiros são movidos enquanto tais condições forem verdadeiras



QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Ponteiro down é movido de lb em direção á ub
 - todo elemento de posição abaixo de down deve ser menor ou igual ao pivô
 - ▣ Ponteiro up é movido de ub em direção á lb
 - todo elemento de posição acima de up deve ser maior ou igual ao pivô
 - ▣ Ponteiros são movidos enquanto tais condições forem verdadeiras

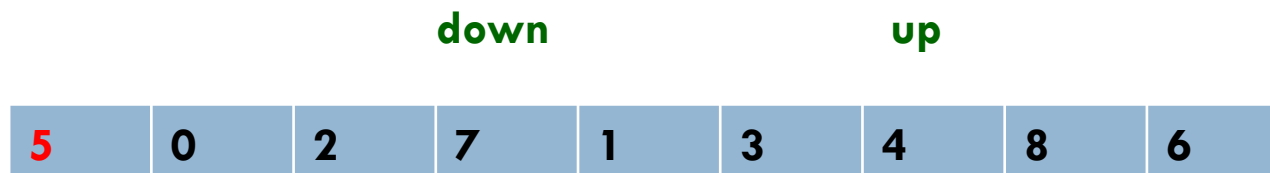
down

up

5	0	2	7	1	3	4	8	6
---	---	---	---	---	---	---	---	---

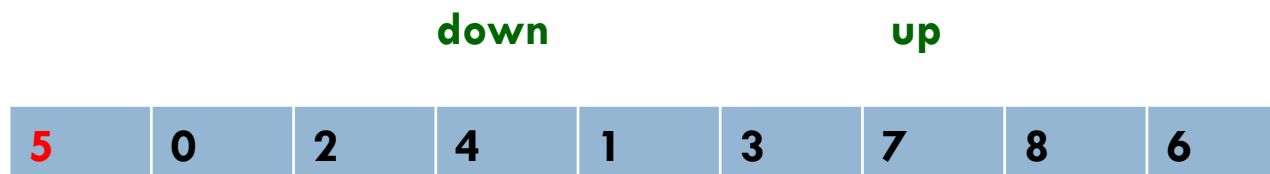
QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Como $up > down$, seus elementos são trocados
 - ▣ Processo prossegue até que $down \geq up$



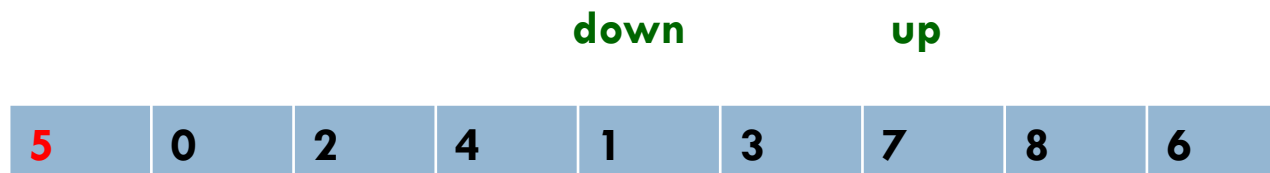
QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Como $up > down$, seus elementos são trocados
 - ▣ Processo prossegue até que $down \geq up$



QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Como $up > down$, seus elementos são trocados
 - ▣ Processo prossegue até que $down \geq up$



QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Como $up > down$, seus elementos são trocados
 - ▣ Processo prossegue até que $down \geq up$

down up

5	0	2	4	1	3	7	8	6
---	---	---	---	---	---	---	---	---

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Como $up > down$, seus elementos são trocados
 - ▣ Processo prossegue até que $down \geq up$

down
up

5	0	2	4	1	3	7	8	6
---	---	---	---	---	---	---	---	---

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Como $up > down$, seus elementos são trocados
 - ▣ Processo prossegue até que $down \geq up$

up down

5	0	2	4	1	3	7	8	6
---	---	---	---	---	---	---	---	---

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Todos os elementos entre as posições $lb+1$ e up são menores que o pivô
 - ▣ Assim, trocamos os elementos das posições up e lb

					up	down		
5	0	2	4	1	3	7	8	6

QuickSort

- Posicionamento do pivô (particionamento)
 - ▣ Todos os elementos entre as posições $lb+1$ e up são menores que o pivô
 - ▣ Assim, trocamos os elementos das posições up e lb

3	0	2	4	1	5	7	8	6
---	---	---	---	---	---	---	---	---

QuickSort

□ Implementação

```
void troca(int *e1, int *e2)
{
    int aux;
    aux = *e1;
    *e1 = *e2;
    *e2 = aux;
}
```

QuickSort

□ Implementação

```
void quicksort(int v[], int lb, int ub)
{
    if(ub<=lb || v == NULL)
        return;

    int pivo = v[lb];
    int down = lb;
    int up= ub;
```

QuickSort

□ Implementação

```
while(up > down)
{
while(down <= ub && v[down] <= pivo)
    down++;
while(up >= lb && v[up] > pivo)
    up--;

if(up>down)
troca(&v[up],&v[down]);
}
```

QuickSort

□ Implementação

```
troca(&v[up], &v[lb]);  
quicksort(v, lb, up - 1);  
quicksort(v, up + 1, ub);  
}
```

QuickSort

□ Análise

- Pior caso: subvetores possuindo sempre tamanho 0 e $n-1$
 - $O(n^2)$
- Melhor caso: subvetores sempre com tamanho $\frac{n}{2}$ e $\frac{n}{2} - 1$
 - $O(n \log(n))$
- Caso médio
 - Tende a se aproximar mais de $O(n \log(n))$
 - Podem ser forçadas divisões na metade do vetor ao selecionar mediana como pivô
 - Porém, com custo extra da seleção da mediana

Referências

1. Introduction to Algorithms, Third Edition, Thomas H. Cormen
2. Robert Sedgewick and Kevin Wayne.
2011. *Algorithms* (4th ed.). Addison-Wesley Professional

